Timur Shakirov

Fullstack developer. Language models inside working products.

20 · Saint Petersburg · remote   +7 981 691-94-36   edtimyr@gmail.com   @TimaxLacs

github.com/TimaxLacs   github.com/TimaxHack   github.com/timaxlax   npmjs.com/~timax

SUMMARY

Engineer who puts language models into working products. Designed and wrote the first version of an LLM gateway - provider routing, fallback, token billing - and every request of a commercial AI aggregator goes through it. For the past year I have been running an e-commerce platform under contract: storefront, catalogue, bilingual product cards, an ingestion pipeline with generation queues, deployment.

I am interested in orchestrating and embedding neural networks - language models and computer vision - into projects and everyday tasks, including automation.

Education: 11 years of Russian general school, no degree in the field. I have been programming since 2021 and since then have worked continuously with different software teams on separate projects - eight teams so far. I have been taking commercial projects since 2024; everything below was built in practice and can be checked in the repositories.

SKILLS

LANGUAGES	Python, TypeScript, JavaScript
FRONTEND	React, Vite, Tailwind, Capacitor (iOS and Android)
BACKEND	Node.js, Express, FastAPI, GraphQL (Hasura), REST, PostgreSQL / PostGIS, Redis, SQLite, MinIO
AI	provider routing and fallback, token billing, text and image generation queues, RAG and MCP, speech recognition
PRODUCTION	Docker, nginx, systemd, Linux, Git, CI, DNS and mail setup
ENGLISH	technical, minimal level

WORKING WITH MODELS

Since 2024 language models have been part of daily development. Current environments: Cursor, Claude Code, ClawCode. I use autocomplete constantly. I fix research, constraints and acceptance criteria myself first, then fill in the solution myself or with a model.

Before working with a model I write a full set of rules into the repository: how to run checks, what not to touch. The same constraints go into CI and hooks so they cannot be skipped. Code written with a model follows narrow steps and separate subagents: plan, edit, run tests and linters.

The cycle is run by agents from cursor-agent-workflows : analysis, plan, implementation and verification are split; verification runs separately and does not accept the code the implementer just wrote. File scope is sealed in advance; if I make parallel edits, I isolate them. Tests the model wrote together with the implementation are not enough: I check negative and edge cases, secrets and dependencies. I always review the resulting code myself and do not hand it off unreviewed.

WORK EXPERIENCE

Deep.Assistant - AI model aggregator — 2024-2025

Founder, backend and DevOps · team of three · @DeepGPTBot, channel @gptDeep - 1,553 subscribers

A commercial product in operation: chat with models, image, music and voice generation, an API gateway, mini-applications in Telegram and VK.

> **LLM gateway - github.com/deep-assistant/api-gateway**
>
> Designed and wrote the first version; the gateway grew out of resale-chatgpt-azure into the infrastructure that carries every product request. Main author of the HTTP server, model configuration, token manager and database layer; I maintain the repository. About 37 models from six providers behind a single OpenAI-compatible interface.
>
> - **A degradation table instead of a plain retry:** each model has a chain of 12-16 substitutes chosen so that the answer stays usable for the same scenario. One unavailable provider does not stop the product.
> - **One account across incomparable price lists:** every model has its own conversion factor from tokens into the internal currency, from 1.7 to 40. The product knows the cost of a request and its margin.
> - **Idempotent transfers between accounts:** exclusive locks with stale-lock detection, atomic writes, recovery of unfinished operations on start-up.

> - **Time to production for new models:** o1-preview and o1-mini on 27 September 2024, deepseek-reasoner on 30 January 2025, each with pricing and fallback from day one.

> #### Telegram bot - github.com/deep-assistant/telegram-bot
> My parts: transfers of the internal currency between users built as a state machine (written from scratch), a user synchronisation service with limited concurrency to stay inside Telegram rate limits, audio transcription with its own pricing factor, payments via Telegram Stars, separated TEST and PROD environments.

E-commerce platform (client project, under NDA)

December 2025 - present

Contract work: frontend, geo module, mobile builds, adjacent backend and deployment

A marketplace with a catalogue and maps: React, TypeScript, Hasura GraphQL, eight services, PostgreSQL / PostGIS, Redis, MinIO, Docker, Capacitor. I run the client application and take part in design decisions; on the backend I own payments, balance and event handlers. Product details are under NDA, the modules I can discuss freely.

- **Storefront:** catalogue, product page, search with autocomplete, filters, registration of several account types with MFA, a UI component library.
- **RU/EN bilingual catalogue and automatic translation of product cards:** dictionaries moved out of components, translation runs through a hook, a server route and a CLI script that processes the catalogue in batches.
- **Catalogue ingestion pipeline:** sources behind a shared contract, deduplication by external id, barcode and attribute fingerprint. Models run after collection in separate queues - text and images. Card facts are never rewritten by a model, otherwise deduplication breaks; a failed worker leaves the product on the storefront with its source data.
- **Wallet and orders:** balance and payment services, external acquiring, status change handlers. Capacitor builds for iOS and Android, deployment through Docker and systemd.

> #### Geo module
> A single interface over Yandex Maps, Google Maps and 2GIS. Map, marker and zone components know nothing about the provider: engines accept a command and pass it to an adapter, the adapter translates unified data into calls of its own SDK. Switching provider is one configuration parameter. Zone geometry: buffers along a route with transitions into circular marker zones through quadratic Bézier curves, polygon union with holes and multipolygons, validation, caching of computations. Draggable markers and user-defined zones; changes made on the map propagate into application state through callbacks. Geocoding on a server route so that API keys never reach the browser. Geometry stored in PostGIS, radius search, smoke tests, documentation for extracting the module into a standalone application.

ManyAir - international money transfer platform

August 2025 - May 2026

Client project: frontend, adjacent backend and server maintenance

Microservices around Hasura GraphQL: router, event handler, messaging, Telegram bot, document generator, currency rates; PostgreSQL, Redis, MinIO. I was responsible for reliable notification delivery (when a channel is unavailable the status returns "not sent" and the invitation is not marked as delivered), data isolation between roles, service deployment, and mail and DNS setup for the project.

Matreshka - public website of an AI/e-commerce platform

2026

Frontend and launch of the website: React, Vite, TypeScript, Tailwind; nginx with SSL, DNS and mail. Second project for the same client - matreshka.club.

AI CASES · 2021-2026

- **Content pipeline with separate AI review.** One model writes the draft, a second one reviews it in a separate call, remarks go back for revision for up to three rounds, then the text is published; state is kept in SQLite to survive a process crash. Generation is separated from review, and a failure does not lose data - ai-post-generator.
- **Reasoning pipeline on top of ordinary chat models**, April 2025. Five stages: intent prediction, step-by-step solution, adversarial critique of its own answer, revision, synthesis; a variant on LangChain with Tree of Thoughts. No built-in reasoning mode is used - the behaviour is reproduced by prompt orchestration - test-resorning.
- **My own tooling for working with models.** Self-hosted LightRAG and an MCP server for it: search modes, reranking, answers with references (lightrag-mcp). 1,900 lines of agent workflow specifications: a safe code change pipeline of five agents, project bootstrap, a research process (cursor-agent-workflows). I use both daily.
- **ai-lector**, 2025 - a bot that rewrites audio lectures into the required format. I built a local Telegram Bot API server with cmake to bypass the file size limit.
- **vk-tg-parser**, 2026 - a pip package: a single parser for Telegram via MTProto and VK via API, with a shared data model and real-time monitoring.
- **Computer vision**, 2021 - a bot for Lake Onega ice cover: freeze level from street-camera frames - github.com/TimaxHack/ghj.

OPEN SOURCE · SINCE 2022

deep.foundation

2022 - present

Organisation member · github.com/deep-foundation

An associative storage ecosystem. AI tooling packages in the official scope: `@deep-foundation/ai`, `ai-models`, `ai-templates`, `ai-tasks` and neighbouring packages. My own package deep-files-sync (13 versions) projects a link graph onto the file system: the contents of the database can be read and edited with `grep`, an editor and git.

- **InnerEcho** - a local voice assistant: speech recognition, a language model, synthesis with voice cloning; npm synthesis clients `xtts-v2-js` and `zonosjs` - InnerEcho.
- **tg-runner** - orchestration of Telegram bots: a state machine with Redis, a Docker worker, SDK and CLI - tg-runner-orchestrator.

---

26 npm packages - npmjs.com/~timax · full portfolio - timaxlacs.github.io